

Exp 13: Write the python program to implement Minimax algorithm for gaming.

Program:

Initial board

```
board = [  
    [' ', ' ', ' '],  
    [' ', ' ', ' '],  
    [' ', ' ', ' ']  
]
```

Print the board

```
def print_board(board):  
    for row in board:  
        print(" | ".join(row))  
        print("-" * 5)
```

Check winner

```
def check_winner(board):  
    # Rows  
    for row in board:  
        if row[0] == row[1] == row[2] != ' ':  
            return row[0]
```

Columns

```
for col in range(3):  
    if board[0][col] == board[1][col] == board[2][col] != ' ':
```

```
    return board[0][col]
```

```
# Diagonals
```

```
if board[0][0] == board[1][1] == board[2][2] != ' ':
```

```
    return board[0][0]
```

```
if board[0][2] == board[1][1] == board[2][0] != ' ':
```

```
    return board[0][2]
```

```
return None
```

```
# Check if board is full
```

```
def is_full(board):
```

```
    for row in board:
```

```
        if ' ' in row:
```

```
            return False
```

```
    return True
```

```
# Minimax Algorithm
```

```
def minimax(board, depth, is_max):
```

```
    winner = check_winner(board)
```

```
    if winner == 'O':
```

```
        return 10 - depth
```

```
    elif winner == 'X':
```

```
        return depth - 10
```

```
elif is_full(board):
```

```
    return 0
```

```
if is_max:
```

```
    best = -math.inf
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            if board[i][j] == '':
```

```
                board[i][j] = 'O'
```

```
                best = max(best, minimax(board, depth + 1, False))
```

```
                board[i][j] = ''
```

```
    return best
```

```
else:
```

```
    best = math.inf
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            if board[i][j] == '':
```

```
                board[i][j] = 'X'
```

```
                best = min(best, minimax(board, depth + 1, True))
```

```
                board[i][j] = ''
```

```
    return best
```

```
# Find best move
```

```
def best_move(board):
```

```
    best_score = -math.inf
```

```
    move = (-1, -1)
```

```
for i in range(3):
    for j in range(3):
        if board[i][j] == ' ':
            board[i][j] = 'O'
            score = minimax(board, 0, False)
            board[i][j] = ' '

            if score > best_score:
                best_score = score
                move = (i, j)

return move
```

Example Board

```
board = [
    ['X', 'O', 'X'],
    ['O', 'X', ' '],
    [' ', ' ', 'O']
]
```

```
print("Current Board:")
```

```
print_board(board)
```

```
move = best_move(board)
```

```
print("\nBest Move for AI (O):", move)
```

```
board[move[0]][move[1]] = 'O'
```

```
print("\nBoard After AI Move:")
```

```
print_board(board)
```

Output:

Current Board:

```
X|O|X
```

```
-----
```

```
O|X|
```

```
-----
```

```
| |O
```

```
-----
```

Best Move for AI (O): (2, 0)

Board After AI Move:

```
X|O|X
```

```
-----
```

```
O|X|
```

```
-----
```

```
O| |O
```

```
-----
```

Result:

Hence, the simple the python program to implement Minimax algorithm for gaming is done

Exp14 : Write the Python Program to implement Alpha & Beta pruning algorithm for gaming.

Program:

Python Program to Implement Alpha-Beta Pruning Algorithm

```
import math
```

```
# Terminal node values
```

```
values = [3, 5, 6, 9, 1, 2, 0, -1]
```

```
# Alpha-Beta Pruning Function
```

```
def alpha_beta(depth, nodeIndex, maximizingPlayer, values, alpha, beta):
```

```
    # Leaf node reached
```

```
    if depth == 3:
```

```
        return values[nodeIndex]
```

```
    if maximizingPlayer:
```

```
        best = -math.inf
```

```
    for i in range(2):
```

```
        value = alpha_beta(depth + 1,
```

```
                           nodeIndex * 2 + i,
```

```
                           False,
```

```
                           values,
```

```
                           alpha,
```

```
                           beta)
```

```

    best = max(best, value)
    alpha = max(alpha, best)

    # Beta Cutoff
    if beta <= alpha:
        break

    return best

else:
    best = math.inf

    for i in range(2):
        value = alpha_beta(depth + 1,
                            nodeIndex * 2 + i,
                            True,
                            values,
                            alpha,
                            beta)

        best = min(best, value)
        beta = min(beta, best)

    # Alpha Cutoff
    if beta <= alpha:
        break

```

```
return best
```

```
# Driver Code
```

```
result = alpha_beta(0, 0, True, values, -math.inf, math.inf)
```

```
print("Optimal value using Alpha-Beta Pruning is:", result)
```

Output:

Optimal value using Alpha-Beta Pruning is: 5

Result:

Hence the python program to implement Apha & Beta pruning algorithm for gaming is done.

Exp15 :Write the python program to implement Decision Tree

Program:

Python Program to Implement Decision Tree

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import accuracy_score
```

Sample Dataset

```
X = [  
    [22, 1],  
    [25, 0],  
    [47, 1],  
    [52, 0],  
    [46, 1],  
    [56, 0],  
    [28, 1],  
    [35, 0]  
]
```

Target Labels

1 = Buy, 0 = Not Buy

```
y = [1, 1, 0, 0, 0, 0, 1, 1]
```

Split the dataset

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y, test_size=0.25, random_state=42  
)
```

```
# Create Decision Tree Classifier
```

```
model = DecisionTreeClassifier()
```

```
# Train the model
```

```
model.fit(X_train, y_train)
```

```
# Predict the test data
```

```
y_pred = model.predict(X_test)
```

```
# Print results
```

```
print("Predicted Values :", y_pred)
```

```
print("Actual Values  :", y_test)
```

```
# Accuracy
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy :", accuracy)
```

Output:

Predicted Values : [1 0]

Actual Values : [1, 0]

Accuracy : 1.0

Result:

Hence the python program to implement Decision Tree is done.

Exp 16: Write the python program to implement Feed forward neural Network

Program:

```
# Python Program to Implement Feed Forward Neural Network
```

```
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
# Sample Dataset
```

```
X = [
    [2, 3],
    [1, 5],
    [2, 8],
    [5, 7],
    [6, 9],
    [7, 3],
    [8, 5],
    [9, 6]
]
```

```
# Target Labels
```

```
# 0 = Class A, 1 = Class B
```

```
y = [0, 0, 0, 1, 1, 1, 1, 1]
```

```
# Split the dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y, test_size=0.25, random_state=42
)
```

```
# Create Feed Forward Neural Network
```

```
model = MLPClassifier(
    hidden_layer_sizes=(5,),
    max_iter=1000,
    random_state=42
)
```

```
# Train the model
```

```
model.fit(X_train, y_train)
```

```
# Predict the test data
```

```
y_pred = model.predict(X_test)
```

```
# Display Results
```

```
print("Predicted Values :", y_pred)
```

```
print("Actual Values  :", y_test)
```

```
# Calculate Accuracy
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy :", accuracy)
```

Output:

```
Predicted Values : [1 0]
```

```
Actual Values  : [1, 0]
```

```
Accuracy : 1.0
```

Result: The above program was executed and output was verified.

